

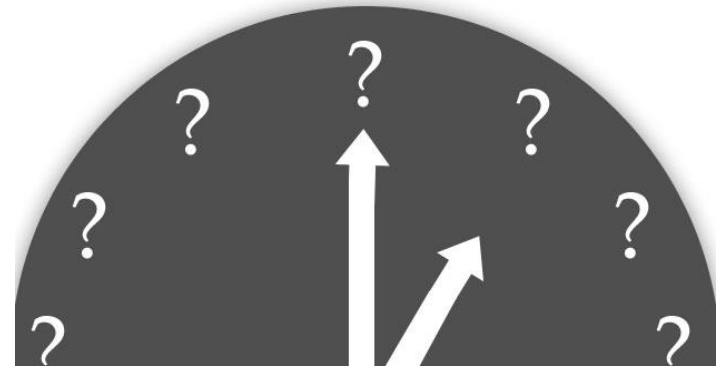
What is Unit testing?

Unit tests are **whitebox** tests written by **developers**, and designed to **verify small units** of program functionality.



When to write unit tests?

Unit tests are written by you, the **developer**, **concurrently** with implementation.



Good Unit Testing

- **Automatic:** Run completely by itself, without any human input.
- **Atomic:** Determine by itself whether the function it is testing has passed or failed, without a human interpreting the results
- **Single responsibility:** Test exactly one feature
- **Independent:** Run in isolation, separate from any other test cases (even if they test the same functions)
- **Repeatable:** Multiple invocations of the test should consistently return the same value.

JUnit

- Unit testing framework for the Java Programming Language.
- Open source (<http://JUnit.org>)
- Framework for both writing and automated execution of unit tests
- Can be integrated with eclipse.

The logo for JUnit, featuring the word "JUnit" in a serif font. The "J" is green, and the "Unit" is red.

JUnit in Action

```
package edu.siu.cs435;

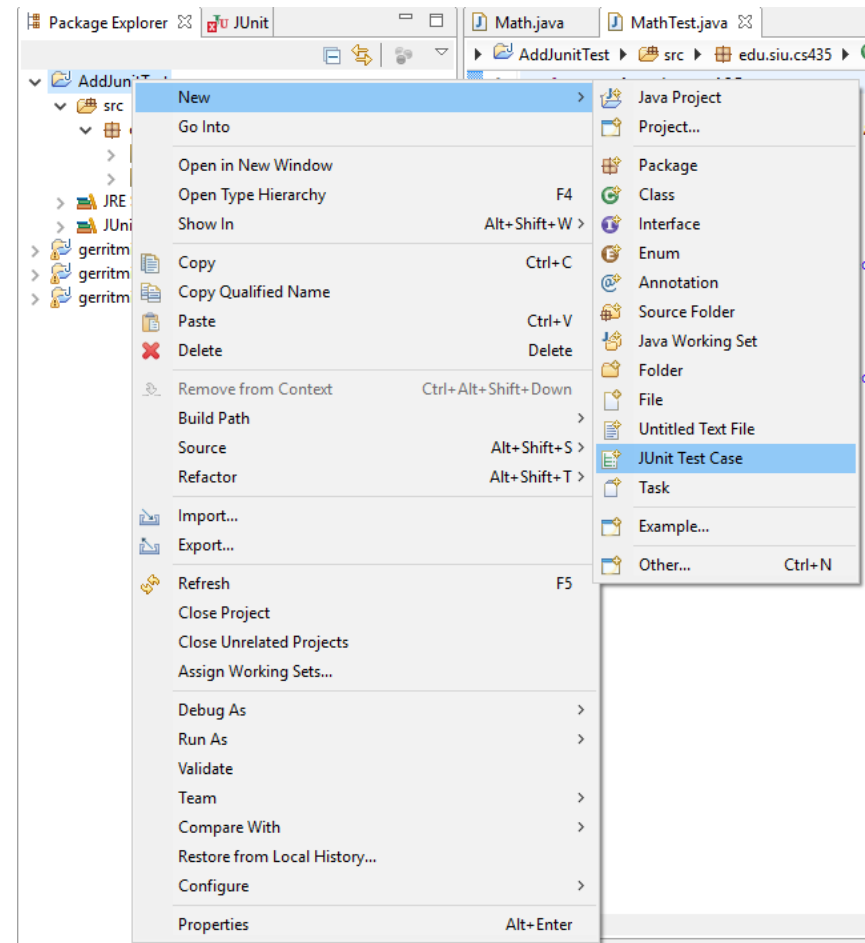
public class Math
{
    public int add(int a, int b) {
        return a + b;
    }

    public int sub(int a, int b) {
        return a - b;
    }
}
```

JUnit and Eclipse

- To add JUnit to an Eclipse project, click:
 - **Project** → **Properties** → **Build Path** → **Libraries** → **Add Library...** → **JUnit** → **JUnit 4** → **Finish**

- To create a test case:
 - right-click a file and choose **New** → **Test Case**
 - or click **File** → **New** → **JUnit Test Case**
 - Eclipse can create stubs of method tests for you.



JUnit in Eclipse

New JUnit Test Case

JUnit Test Case

Select the name of the new JUnit test case. You have the options to specify the class under test and on the next page, to select methods to be tested.

☐ New JUnit 3 test ☒ New JUnit 4 test

Source folder:

Package:

Name:

Superclass:

Which method stubs would you like to create?

☐ setUpBeforeClass() ☐ tearDownAfterClass()
☐ setUp() ☐ tearDown()
☐ constructor

Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments

Class under test:

New JUnit Test Case

Test Methods

Select methods for which test method stubs should be created.

Available methods:

- ☒ **Math**
 - ☒ add(int, int)
 - ☒ sub(int, int)
- ☐ **Object**
 - ☐ Object()
 - ☐ getClass()
 - ☐ hashCode()
 - ☐ equals(Object)
 - ☐ clone()
 - ☐ toString()
 - ☐ notify()
 - ☐ notifyAll()
 - ☐ wait(long)
 - ☐ wait(long, int)
 - ☐ wait()
 - ☐ finalize()

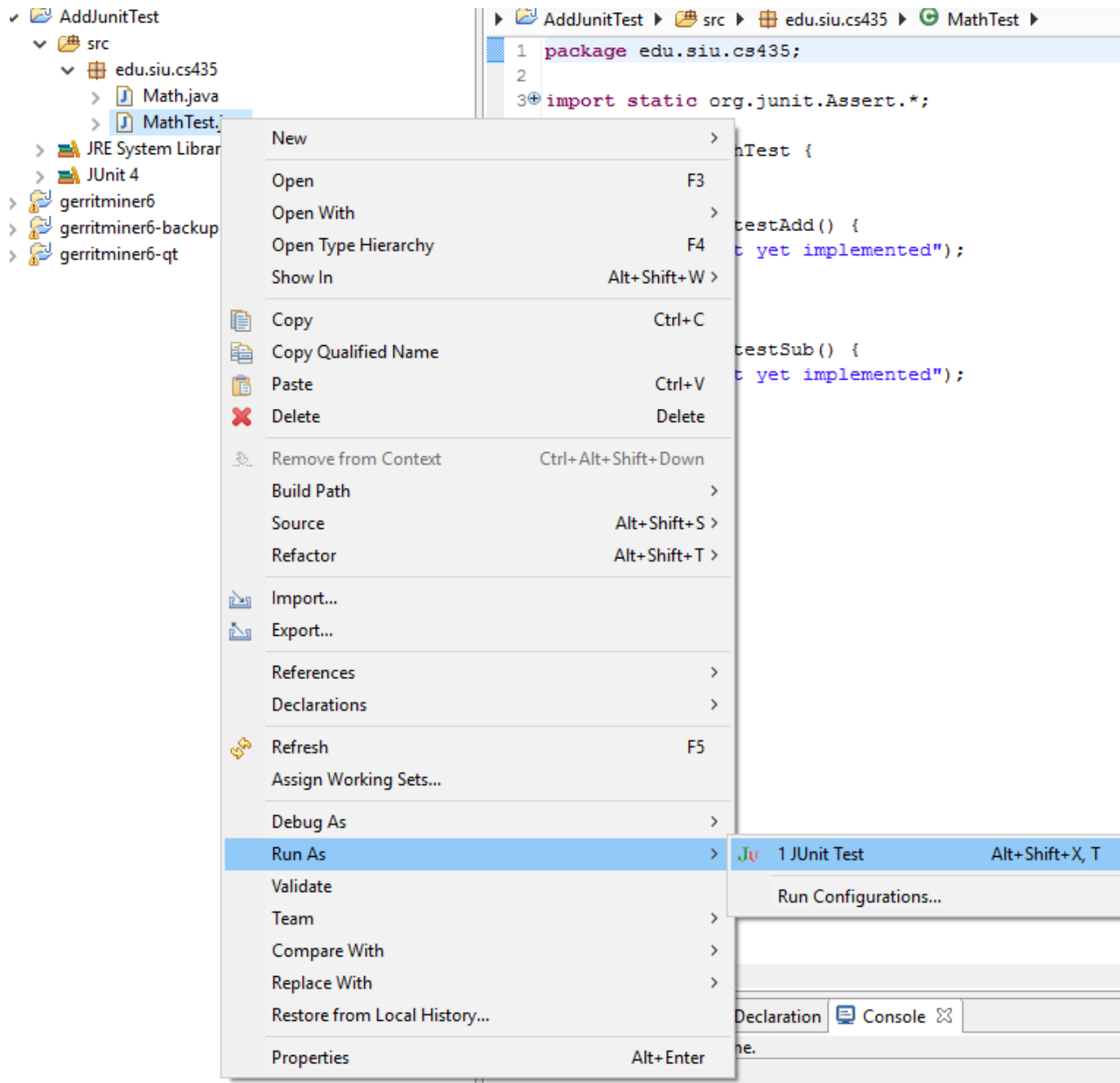
2 methods selected.

☐ Create final method stubs
☐ Create tasks for generated test methods

Math.java MathTest.java

AddJUnitTest src edu.siu.cs435 MathTest

```
1 package edu.siu.cs435;
2
3+ import static org.junit.Assert.*;
6
7 public class MathTest {
8
9-   @Test
10   public void testAdd() {
11       fail("Not yet implemented");
12   }
13
14-   @Test
15   public void testSub() {
16       fail("Not yet implemented");
17   }
18
19 }
20
```

Package Explorer

JUnit

MathTest

Runs: 2/2 Errors: 0 Failures: 2

edu.siu.cs435.MathTest [Runner: JUnit 4] (0.015 s)

- testAdd (0.015 s)
- testSub (0.000 s)

Failure Trace

Math.java MathTest.java

AddJUnitTest > src > edu.siu.cs435 > MathTest

```
1 package edu.siu.cs435;
2
3 import static org.junit.Assert.*;
4
5
6
7 public class MathTest {
8
9     @Test
10    public void testAdd() {
11        fail("Not yet implemented");
12    }
13
14    @Test
15    public void testSub() {
16        fail("Not yet implemented");
17    }
18
19 }
20
```

Package Explorer

JUnit

Finished after 0.022 seconds

Runs: 2/2 Errors: 0 Failures: 0

edu.siu.cs435.MathTest [Runner: JUnit 4] (0.001 s)

Failure Trace

Math.java MathTest.java

AddJUnitTest ▶ src ▶ edu.siu.cs435 ▶ MathTest ▶ testSub(): void

```
1 package edu.siu.cs435;
2
3 import static org.junit.Assert.*;
4
5
6
7 public class MathTest {
8
9     @Test
10    public void testAdd() {
11        Math calc = new Math();
12        assertEquals(11, calc.add(5,6));
13        assertEquals(0, calc.add(0,0));
14        assertEquals(-10, calc.add(-5,-5));
15        assertEquals(1, calc.add(-5,6));
16    }
17
18
19     @Test
20    public void testSub() {
21        Math calc = new Math();
22        assertEquals(-1, calc.sub(5,6));
23        assertEquals(0, calc.sub(0,0));
24        assertEquals(0, calc.sub(-5,-5));
25        assertEquals(-11, calc.sub(-5,6));
26    }
27
28
29 }
30
```

JUnit assertion methods

<code>assertTrue(test)</code>	fails if the boolean test is <code>false</code>
<code>assertFalse(test)</code>	fails if the boolean test is <code>true</code>
<code>assertEquals(expected, actual)</code>	fails if the values are not equal
<code>assertSame(expected, actual)</code>	fails if the values are not the same (by <code>==</code>)
<code>assertNotSame(expected, actual)</code>	fails if the values <i>are</i> the same (by <code>==</code>)
<code>assertNull(value)</code>	fails if the given value is <i>not</i> <code>null</code>
<code>assertNotNull(value)</code>	fails if the given value is <code>null</code>
<code>fail()</code>	causes current test to immediately fail

- Each method can also be passed a string to display if it fails:
 - e.g. `assertEquals("message", expected, actual)`

JUnit Framework

Provides following important features

1. Fixtures
2. Test suites
3. Test runners
4. JUnit classes

Fixtures

- **Fixtures** is a fixed state of a set of objects used as a baseline for running tests. The purpose of a test fixture is to ensure that there is a well known and fixed environment in which tests are run so that results are repeatable.
- **@Before**
 - Performs this function before each test case
 - Since each of the tests are independent, each test will receive *its own instance of whatever is created in the @Before methods*
- **@Test**
 - Indicates individual test cases
 - Must be present to tell JUnit which methods are tests and which are helpers
- **@After**
 - The teardown after each test case
 - Usually need to worry about this only if you have created external resources

Finished after 0.015 seconds

Runs: 2/2 Errors: 0 Failures: 0

> edu.siu.cs435.MathTest [Runner: JUnit 4] (0.000 s)

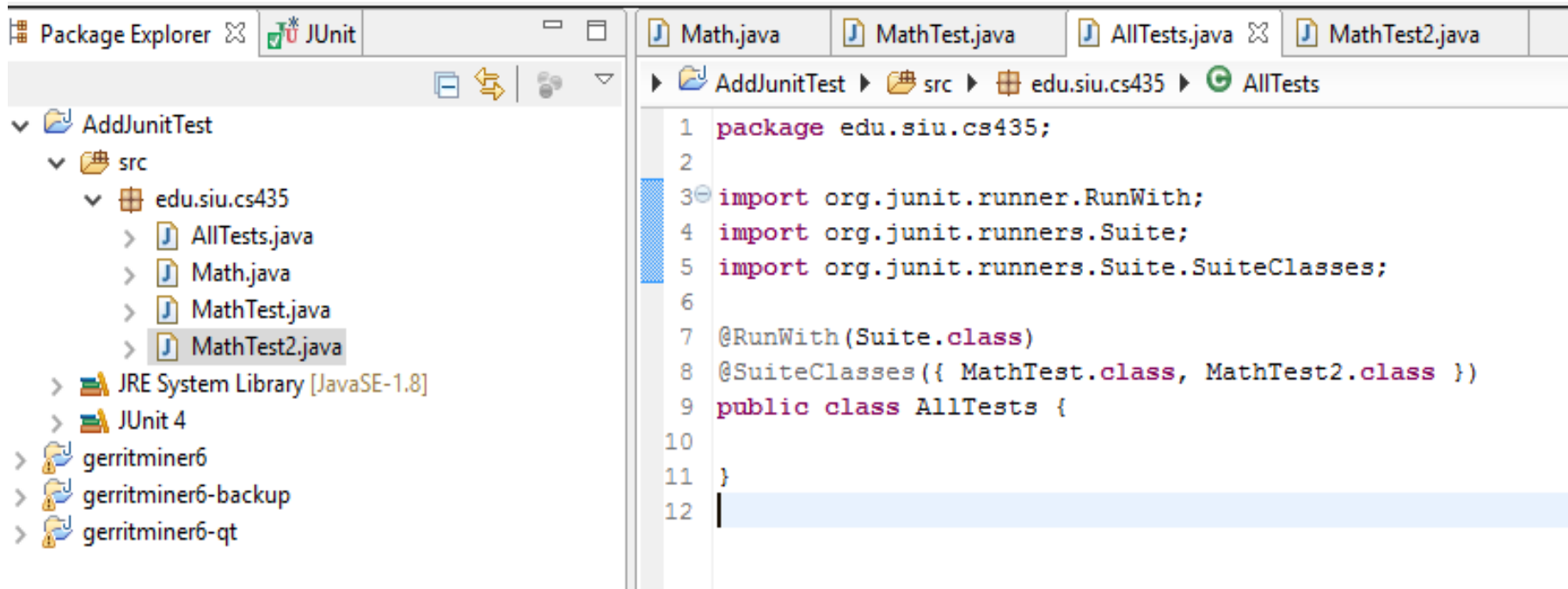
Failure Trace

AddJUnitTest ▶ src ▶ edu.siu.cs435 ▶ MathTest ▶ testSub(): void

```
1 package edu.siu.cs435;
2
3 import static org.junit.Assert.*;
4
5
6
7
8
9 public class MathTest {
10     Math calc;
11
12     @Before
13     public void setUp()
14     {
15         calc = new Math();
16     }
17
18     @Test
19     public void testAdd() {
20         assertEquals(11, calc.add(5,6));
21         assertEquals(0, calc.add(0,0));
22         assertEquals(-10, calc.add(-5,-5));
23         assertEquals(1, calc.add(-5,6));
24     }
25
26
27     @Test
28     public void testSub() {
29         assertEquals(-1, calc.sub(5,6));
30         assertEquals(0, calc.sub(0,0));
31         assertEquals(0, calc.sub(-5,-5));
32         assertEquals(-11, calc.sub(-5,6));
33     }
34
35
36     @After
37     public void tearDown() {
38         calc=null;
39     }
40
41 }
```

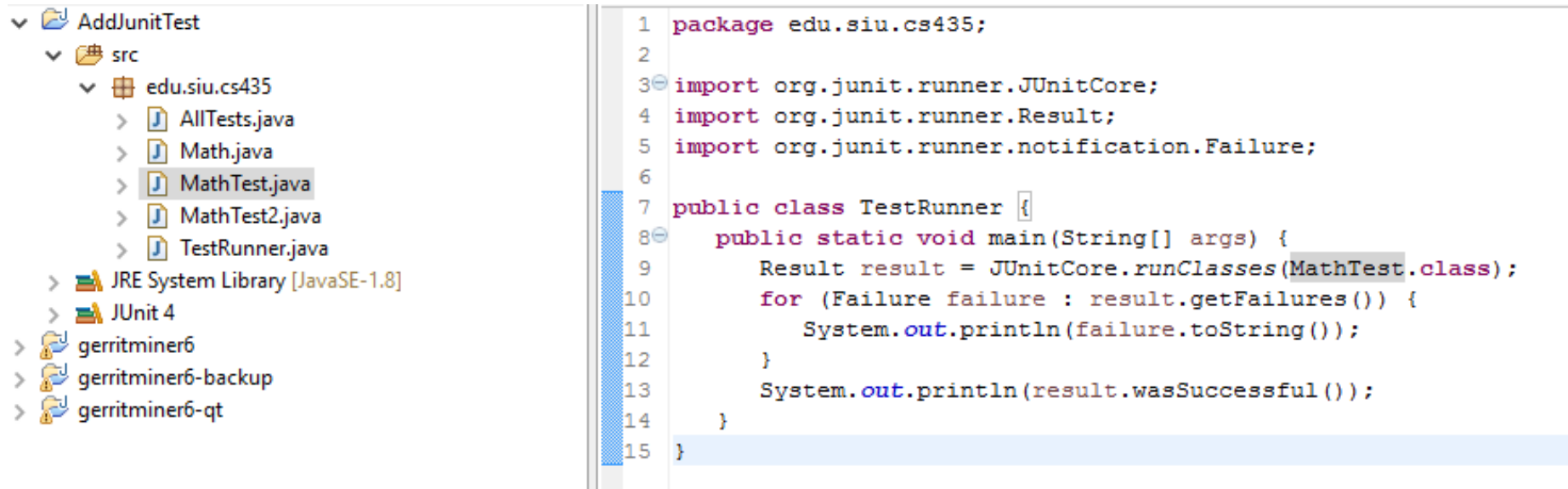
Test Suite

- **Test suite** means bundle a few unit test cases and run it together. In JUnit, both `@RunWith` and `@Suite` annotation are used to run the suite test.



Test Runner

- **Test runner** is used for executing the test cases.



The screenshot displays an IDE interface. On the left, a project tree shows a project named 'AddJUnitTest' with a source folder 'src'. Inside 'src', there is a package 'edu.siu.cs435' containing several Java files: 'AllTests.java', 'Math.java', 'MathTest.java' (highlighted), 'MathTest2.java', and 'TestRunner.java'. Below the project files, there are references to 'JRE System Library [JavaSE-1.8]' and 'JUnit 4'. On the right, the code editor shows the content of 'TestRunner.java'. The code is as follows:

```
1 package edu.siu.cs435;
2
3 import org.junit.runner.JUnitCore;
4 import org.junit.runner.Result;
5 import org.junit.runner.notification.Failure;
6
7 public class TestRunner {
8     public static void main(String[] args) {
9         Result result = JUnitCore.runClasses(MathTest.class);
10        for (Failure failure : result.getFailures()) {
11            System.out.println(failure.toString());
12        }
13        System.out.println(result.wasSuccessful());
14    }
15 }
```

JUnit exercise

Given a `Date` class with the following methods:

```
-public Date(int year, int month, int day)
-public Date() // today
-public int getDay(), getMonth(), getYear()
-public void addDays(int days) // advances by days
-public int daysInMonth()
-public String dayOfWeek() // e.g. "Sunday"
-public boolean equals(Object o)
-public boolean isLeapYear()
-public void nextDay() // advances by 1 day
-public String toString()
```

- Come up with unit tests to check the following:
 - That no `Date` object can ever get into an invalid state.
 - That the `addDays` method works properly.
 - It should be efficient enough to add 1,000,000 days in a call.

What's wrong with this?

```
public class DateTest {  
    @Test  
    public void test1() {  
        Date d = new Date(2050, 2, 15);  
        d.addDays(4);  
        assertEquals(d.getYear(), 2050);  
        assertEquals(d.getMonth(), 2);  
        assertEquals(d.getDay(), 19);  
    }  
  
    @Test  
    public void test2() {  
        Date d = new Date(2050, 2, 15);  
        d.addDays(14);  
        assertEquals(d.getYear(), 2050);  
        assertEquals(d.getMonth(), 3);  
        assertEquals(d.getDay(), 1);  
    }  
}
```

Well-structured assertions

```
public class DateTest {
    @Test
    public void test1() {
        Date d = new Date(2050, 2, 15);
        d.addDays(4);
        assertEquals(2050, d.getYear()); // expected
        assertEquals(2, d.getMonth()); // value should
        assertEquals(19, d.getDay()); // be at LEFT
    }

    @Test
    public void test2() {
        Date d = new Date(2050, 2, 15);
        d.addDays(14);
        assertEquals("year after +14 days", 2050, d.getYear());
        assertEquals("month after +14 days", 3, d.getMonth());
        assertEquals("day after +14 days", 1, d.getDay());
    } // test cases should usually have messages explaining
    // what is being checked, for better failure output
}
```

Expected answer objects

```
public class DateTest {
    @Test
    public void test1() {
        Date d = new Date(2050, 2, 15);
        d.addDays(4);
        Date expected = new Date(2050, 2, 19);
        assertEquals(expected, d);    // use an expected answer
                                     // object to minimize tests
    }

                                     // (Date must have toString
                                     // and equals methods)

    @Test
    public void test2() {
        Date d = new Date(2050, 2, 15);
        d.addDays(14);
        Date expected = new Date(2050, 3, 1);
        assertEquals("date after +14 days", expected, d);
    }
}
```

Naming test cases

```
public class DateTest {
    @Test
    public void test_addDays_withinSameMonth_1() {
        Date actual = new Date(2050, 2, 15);
        actual.addDays(4);
        Date expected = new Date(2050, 2, 19);
        assertEquals("date after +4 days", expected, actual);
    }
    // give test case methods really long descriptive names

    @Test
    public void test_addDays_wrapToNextMonth_2() {
        Date actual = new Date(2050, 2, 15);
        actual.addDays(14);
        Date expected = new Date(2050, 3, 1);
        assertEquals("date after +14 days", expected, actual);
    }
    // give descriptive names to expected/actual values
}
```

What's wrong with this?

```
public class DateTest {  
    @Test  
    public void test_addDays_addJustOneDay_1() {  
        Date actual = new Date(2050, 2, 15);  
        actual.addDays(1);  
        Date expected = new Date(2050, 2, 16);  
        assertEquals(  
            "should have gotten " + expected + "\n" +  
            " but instead got " + actual + "\n",  
            expected, actual);  
    }  
    ...  
}
```

Good assertion messages

```
public class DateTest {  
    @Test  
    public void test_addDays_addJustOneDay_1() {  
        Date actual = new Date(2050, 2, 15);  
        actual.addDays(1);  
        Date expected = new Date(2050, 2, 16);  
        assertEquals("adding one day to 2050/2/15",  
            expected, actual);  
    }  
    ...  
}
```

// JUnit will already show
// the expected and actual
// values in its output;
//
// don't need to repeat them
// in the assertion message

